



UNIVERSIDAD
DON BOSCO

Programación Orientada a Objetos en Java: Herencia y Polimorfismo

Reutilización de clases y especialización de objetos.



Los Pilares de la POO



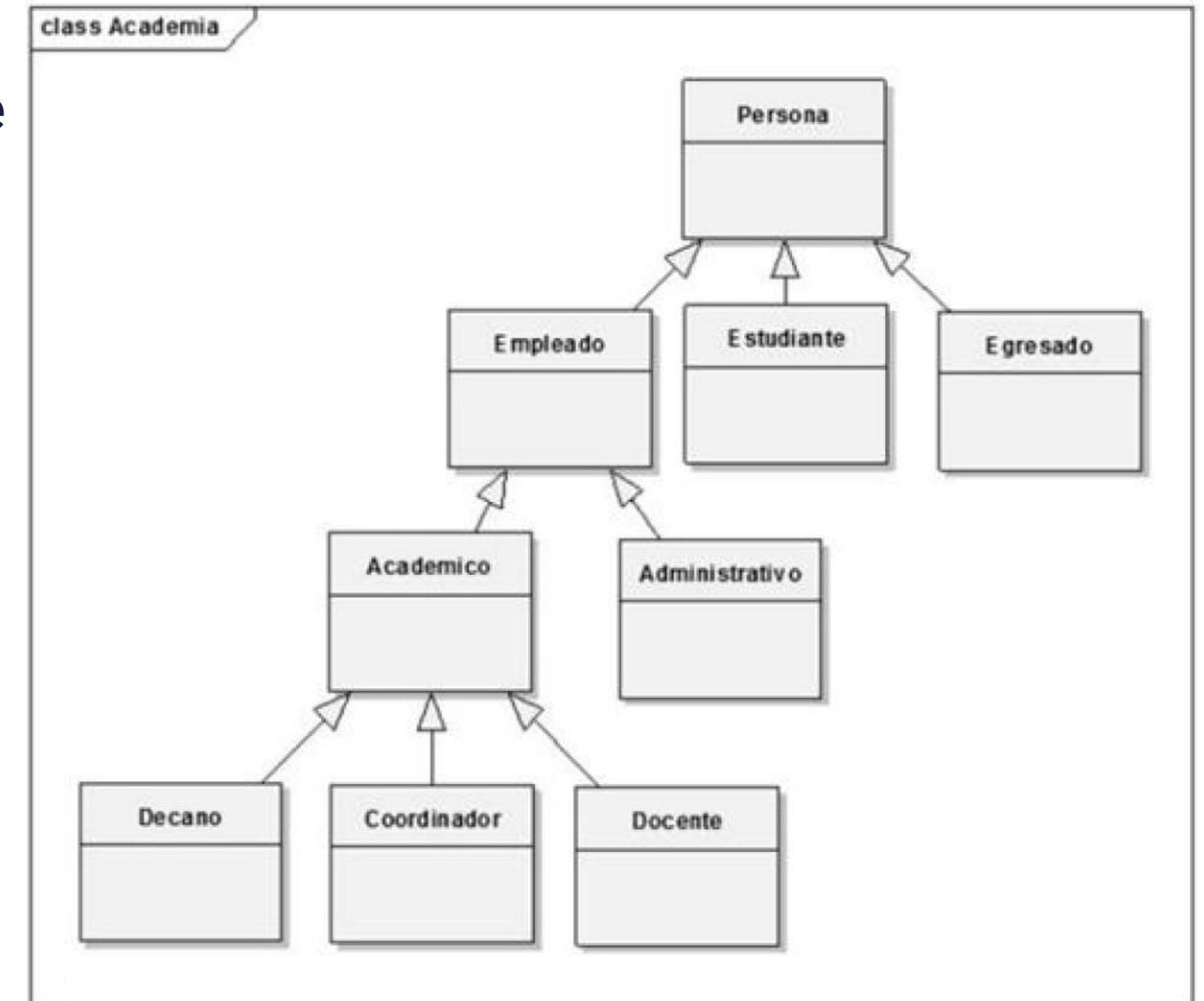
Herencia en Java

- La herencia permite definir nuevas clases basadas en clases existentes, con el fin de reutilizar el código previamente desarrollado, generando una jerarquía de clases dentro de una aplicación.
- Entonces, si una clase se deriva de otra, esta hereda sus atributos y métodos.
- La clase derivada puede añadir nuevos atributos y métodos, así como redefinir los atributos y métodos heredados.
- Para que un atributo y método puedan ser heredados es necesario que su visibilidad sea “public” o “protected”.
- En Java, una clase solo puede derivar de una única clase, con lo cual, no es posible realizar herencia múltiple con base en clases.
- Sin embargo, es posible “simular” la herencia múltiple con base en las interfaces, como se vera mas adelante en este Curso.

Los Pilares de la POO

Herencia en Java

- Un ejemplo del concepto de herencia puede ser considerando, los miembros de una institución de educación:
 - ✓ La institución está conformada por personas, pero cada persona tiene un rol distinto dentro de la institución, que podría ser de empleado, estudiante o egresado.
 - ✓ Así administrativo. De académico se puede derivar, decano, coordinador y docente.
 - ✓ De administrativo se puede derivar de acuerdo a la cantidad de departamentos de la institución.
-
- La representación UML de herencia del caso expuesto aquí en lenguaje de modelado es la siguiente:

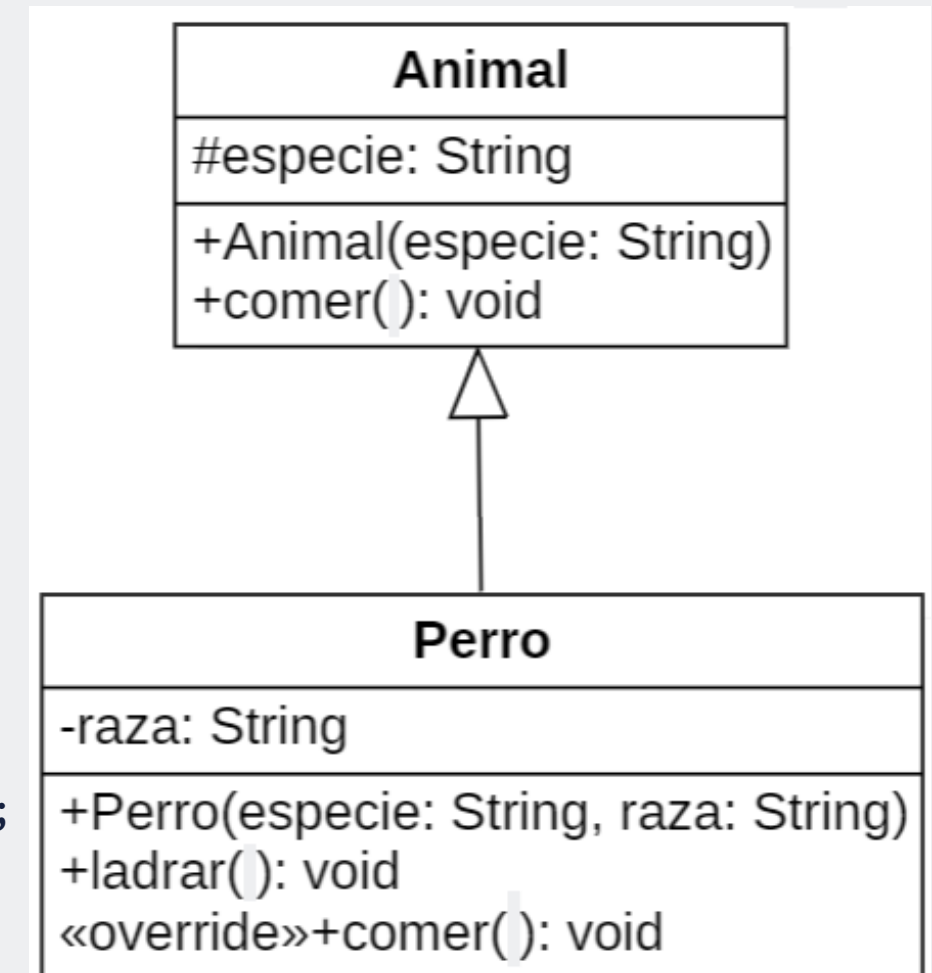


Herencia / Generalización (Inheritance / Generalization)

La herencia es una relación "es-un" (is-a) donde una clase (denominada **subclase** o **clase hija**) deriva o extiende otra clase (**superclase** o **clase padre**), adquiriendo sus atributos y comportamientos. En Java, se utiliza la palabra clave **extends**.

```
// Ejemplo de Herencia en Java
class Animal {
    protected String especie;
    public Animal(String especie) {
        this.especie = especie;
    }
    public void comer() {
        System.out.println("El " + especie + " está comiendo.");
    }
}

class Perro extends Animal { // Perro 'es un' Animal
    private String raza;
    public Perro(String especie, String raza) {
        super(especie); // Llama al constructor de la superclase
        this.raza = raza;
    }
    public void ladrar() {
        System.out.println("El perro de raza " + raza + " está ladrando.");
    }
    @Override // Buena práctica para indicar que se sobrescribe un método
    public void comer() {
        System.out.println("El perro " + raza + " come su ración.");
    }
}
```



Sentencia **super**

- La sentencia **super** es utilizada en una subclase para acceder a métodos implementados en la clase superior en el concepto de herencia.
- Es comúnmente utilizada para acceder al constructor de la clase superior desde el constructor de la clase inferior.
- Por ejemplo, en el caso de jerarquía de herencia de personal académico debe existir una clase persona apellido y correo, y una clase Estudiante que puede acceder a estos atributos pero que adicionalmente, tiene atributos como código y facultad.
- Al implementar el constructor de la clase estudiante para asignar los valores de los atributos, se puede hacer un llamado al constructor de la clase persona en dicha clase.

```
public class Persona{  
  
    protected int id  
    protected String nombre  
    protected String apellido  
    protected String correo  
  
    public Persona(int id, String nombre, String apellido,  
String correo){  
        this.id=id;  
        this.nombre=nombre;  
        this.apellido=apellido;  
        this.correo=correo;  
    }  
}
```

```
public class Estudiante extends Persona{  
  
    private int codigo  
    private String facultad  
  
    public Estudiante(int id, String nombre, String apellido,  
String correo, int codigo, String facultad){  
        super(id, nombre, apellido, correo);  
        this.codigo=codigo;  
        this.facultad=facultad;  
    }  
}
```

Herencia: Sobreescritura de Metodos



Notacion @override

- Bajo la herencia, la sobreescritura de métodos, también llamada redefinición de métodos, permite a una subclase proporcionar una implementación específica para un método heredado de su superclase, usando el mismo nombre, tipo de retorno y parámetros, permitiendo además el polimorfismo.
- Para identificar un método a sobrescribir, se utiliza la anotación **@override**. Esta anotación ayuda al compilador a reconocer métodos que se están sobrescribiendo en una subclase y verificar si el nombre del nuevo metodo coincide con el de la superclase.

Reglas para implementar la redefinicion de un metodo en la subclase:

- @Override: Es una anotación opcional pero recomendada para asegurar que estás sobrescribiendo correctamente; el compilador emitirá una advertencia si no es así.
- Visibilidad: El método sobrescrito no puede ser más restrictivo que el original (por ejemplo, no puede ser privado si el original no lo era).
- Polimorfismo: Permite que un objeto de la subclase se comporte de manera diferente cuando se llama al método, incluso si se trata a través de una referencia de la superclase (ligadura dinámica).

Herencia: Sobreescritura de Metodos

Codificacion en Java Uso de `@override`

Ejemplo de Sobreescritura de Método en Java

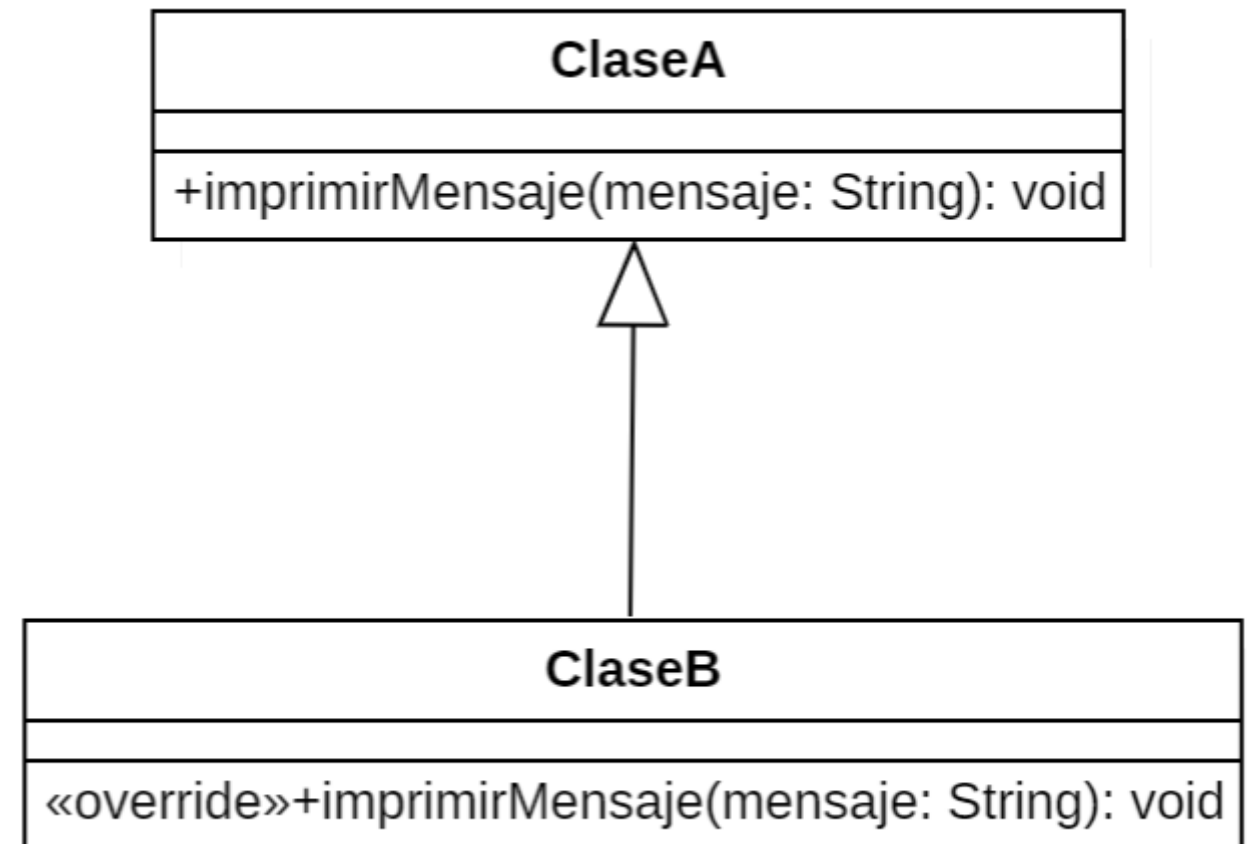
```
public class ClaseA {  
    public void imprimirMensaje(String mensaje) {  
        System.out.println(mensaje);  
    }  
}
```

Método original

```
public class ClaseB extends ClaseA {  
    @override  
    public void imprimirMensaje(String mensaje) {  
        System.out.println("El mensaje es: ");  
        System.out.println(mensaje);  
    }  
}
```

Método sobrescrito

Representacion con UML



Herencia: Interfaces

- Una **interface** es un tipo especial de clase que permite realizar un conjunto de declaraciones de metodos sin implementacion.
- En una interface tambien se pueden definir constantes que, son implicitamente *public*, *static* y *final*; además, deben siempre inicializarse en la declaración.
- Para que una clase use las definiciones de una interface, dicha clase debe incluir la sentencia "implements" la cual indica que implementa la interfacer.
- La sintaxis es la siguiente:

```
public interface MiInterfaz {  
    ...  
}  
  
public class MiClase implements MiInterfaz {  
    ...  
}
```

Herencia: Interfaces



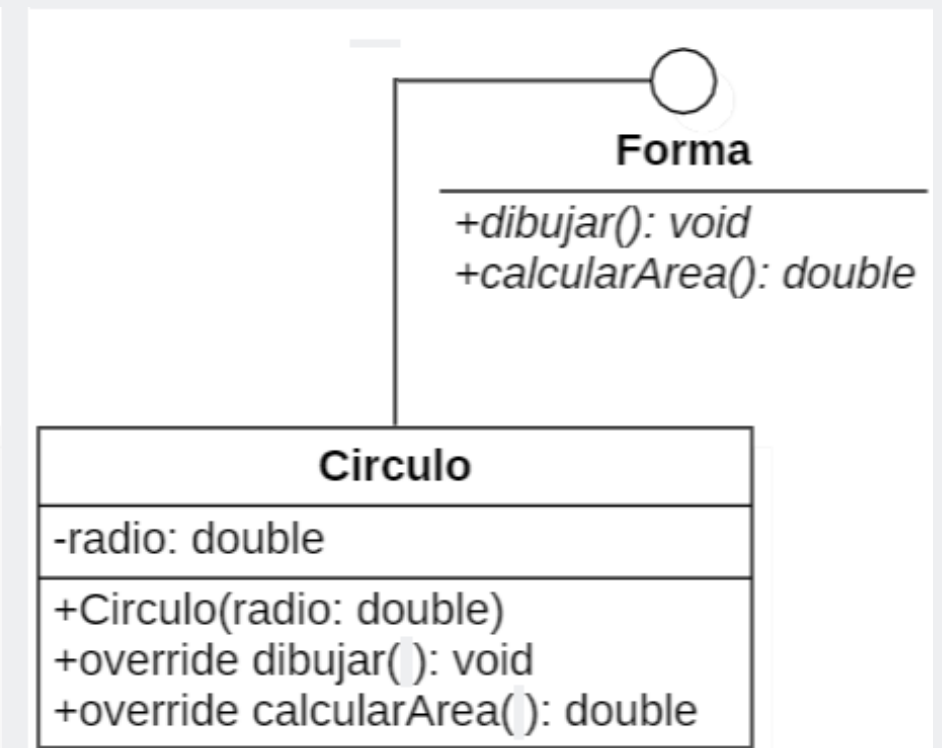
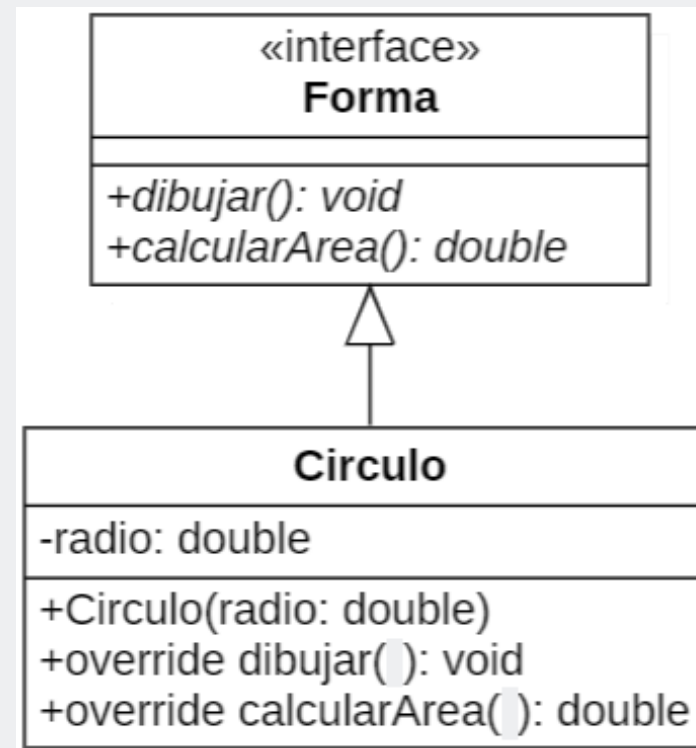
- El objetivo de los métodos declarados en una interfaz es definir un tipo de conducta para las clases que implementan dicha interfaz.
- Todas las clases que colocan en funcionamiento una determinada interfaz, están obligadas a proporcionar una implementación de los métodos declarados en la interfaz adquiriendo un comportamiento.
- Una clase en Java puede implementar una o varias interfaces. Para indicar que una clase implementa más de una interfaz, se ponen los nombres de las interfaces separadas por comas, posterior a incluir la sentencia "implements". La sintaxis es la siguiente:

```
public class MiClase implements MiInterfaz1, MiInterfaz2,  
MiInterfazN {  
    ...  
}
```

Realización (Realization)

La realización es la relación donde una clase implementa un contrato definido por una interfaz. Una clase "realiza" los métodos abstractos declarados en una interfaz.

```
// Ejemplo de Realización en Java
import java.util.List; // List es una interfaz
import java.util.ArrayList; // ArrayList es una clase que implementa a List
interface Forma { // Define un contrato
    void dibujar();
    double calcularArea();
}
// Círculo 'realiza' la interfaz Forma
class Circulo implements Forma {
    private double radio;
    public Circulo(double radio) {
        this.radio = radio;
    }
    @Override
    public void dibujar() {
        System.out.println(
            "Dibujando un círculo de radio " +
            radio);
    }
    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```



Resumen y Mejores Prácticas en Java

Principios de Calidad para Clases Java



1 Diseño Previo y Documentación JavaDoc

Considera diseñar tus clases con diagramas UML antes de codificar. Además, documentar el código Java usando `JavaDoc` para explicar la funcionalidad de clases, métodos y atributos, facilitando la comprensión y el mantenimiento.

2 Encapsulación Rigurosa

Declara los atributos como **private** y proporciona acceso controlado a través de métodos **public** (*getters/setters*). Esto protege la integridad de los datos y permite cambiar la implementación interna sin afectar al código cliente.

3 Constructores Validadores

Asegúrate de que los objetos se creen siempre en un estado válido. Utiliza los constructores para validar los parámetros de entrada y establecer los valores iniciales, evitando objetos con datos inconsistentes.

4 Métodos Cohesivos y Concisos

Cada método debe tener una única responsabilidad bien definida, siguiendo el principio de "Single Responsibility". Métodos pequeños y enfocados son más fáciles de entender, probar y mantener.

5 Sobrecarga de Métodos Clara

Utiliza la sobrecarga (**method overloading**) para proporcionar diferentes versiones de un método que realizan operaciones conceptualmente similares, pero con distintos tipos o número de parámetros.

Errores Comunes a Evitar en Java

- Atributos públicos → Usar siempre `private` con `getters/setters`
- Constructores sin validación → Validar parámetros siempre con excepciones apropiadas
- Métodos "Dios" (gigantes) → Dividir en métodos más pequeños y enfocados
- Nombres poco descriptivos → Usar nombres significativos y seguir convenciones (camelCase, PascalCase)
- Ignorar modificadores de acceso → Usar el más restrictivo (`private`) posible por defecto

Siguiente Paso en tu Aprendizaje de Java

Ahora que dominas los fundamentos de las clases en Java, estás listo para explorar conceptos avanzados:

- **Herencia** (`extends`): Reutilización de código mediante jerarquías de clases.
- **Polimorfismo**: Flexibilidad en el comportamiento de objetos a través de métodos sobrescritos.
- **Interfaces** (`interface`): Contratos para implementar comportamientos específicos.
- **Clases Abstractas** (`abstract class`): Plantillas para especializaciones con métodos abstractos y concretos.
- **Manejo de Excepciones** (`try-catch`): Gestionar errores de forma robusta.
- **Colecciones (API Collections)**: Estructuras de datos como `ArrayList`, `HashMap`, `Set`.

Ejercicios Prácticos: Clases y Relaciones en Java POO

Pon a prueba tus conocimientos con estos ejercicios progresivos diseñados para consolidar tu comprensión de las clases, objetos, modificadores de acceso y relaciones en Java.



Ejercicio 1: Creación de Clase Básica

Crea una clase `Libro` con los siguientes atributos: `titulo` (String) y `autor` (String).

Implementa un método llamado `mostrarInformacion()` que imprima el título y el autor del libro.

```
// Clase Libro
public class Libro {
    // Atributos
    String titulo;
    String autor;
    // Método para mostrar información
    public void mostrarInformacion() {
        // Implementar aquí
    }
}

// En la clase principal (Main.java)
public class Main {
    public static void main(String[] args) {
        // Crear un objeto Libro y probar el método

    }
}
```



Ejercicio 2: Constructores y Sobrecarga

Extiende la clase `Libro` del ejercicio anterior. Añade un constructor predeterminado y un constructor parametrizado que inicialice el `titulo` y el `autor`.

Además, sobrecarga el método `mostrarInformacion()` para que acepte un parámetro booleano `conDetalles`, y si es `true`, también imprima un mensaje adicional.

```
// Clase Libro (continuación)
public class Libro {
    String titulo;
    String autor;
    public Libro() { // Constructor predeterminado
        // Inicialización por defecto
    }
    public Libro(String titulo, String autor) { // Constructor parametrizado
        // Inicializar atributos
    }
    public void mostrarInformacion() { // Método mostrarInformacion
        System.out.println("Título: " + this.titulo + ", Autor: " + this.autor);
    }
    // Método mostrarInformacion sobrecargado
    public void mostrarInformacion(boolean conDetalles) {
        // Implementar lógica aquí
    }
}

public class Main {
    public static void main(String[] args) {
        Libro libro1 = new Libro(); // Usar constructor predeterminado
        // Uso constructor parametrizado
        Libro libro2 = new Libro(
            "Cien Años de Soledad", "Gabriel García Márquez");
        libro2.mostrarInformacion(true); // Probar método sobrecargado
    }
}
```